

SQL

PLACEMENT NOTES

Beginner to Advanced | Theory · Syntax · Examples · Optimization · Interview Prep

Prepared for Engineering Students

Target Companies: Product & Service-based (Microsoft, Amazon, TCS, Infosys, Deloitte, and more)

Created by Ansh Banga

LinkedIn: <https://www.linkedin.com/in/ansh-banga-775275330/>

Table of Contents

Table of Contents.....	2
How to Use These Notes.....	4
Sample Schema Used Throughout This Document.....	4
SQL Learning Roadmap — One-Page View	5
Chapter 1 — SQL Fundamentals.....	6
1.1 What Is SQL?	6
1.2 Basic SELECT Syntax	6
1.3 Filtering Data.....	7
Chapter 2 — Joins	9
Chapter 3 — Aggregation: GROUP BY & HAVING.....	11
Chapter 4 — Subqueries	12
Chapter 5 — Common Table Expressions (CTE).....	13
Chapter 6 — Window Functions.....	15
Chapter 7 — Indexes.....	17
7.1 What Is an Index?	17
Chapter 8 — Transactions & ACID.....	19
8.1 ACID Properties.....	19
8.2 Isolation Levels & Concurrency Problems.....	19
8.3 Locking Deadlock Example	20
Chapter 9 — Query Optimization.....	21
9.1 Reading an Execution Plan	21
9.2 High-Impact Optimization Techniques.....	21
Chapter 10 — Normalization (Quick Reference)	23
Chapter 11 — Common Mistakes Checklist.....	24
Chapter 12 — Interview Questions & Answers	25
Chapter 13 — Practice Problems.....	28
Foundational	28
Intermediate	28
Advanced (Window Functions & CTEs)	28
Chapter 14 — Top 25 SQL Functions Cheat Sheet	30
Chapter 15 — Most Asked SQL Interview Patterns	32
Chapter 16 — 30 One-Line SQL Interview Facts	34
Chapter 17 — MySQL vs PostgreSQL vs SQL Server.....	36
Chapter 18 — Common SQL Interview Mistakes (Beyond the Query Itself)	37
Chapter 19 — Placement Revision Checklist	38

Appendix — One-Page Cheat Sheet	39
--	-----------

How to Use These Notes

These notes are structured to take you from SQL fundamentals to advanced, interview-ready mastery. Every chapter follows the same format so you can revise quickly the night before an interview:

- Theory — the concept explained simply, with the 'why' behind it.
- Syntax — the general SQL template you can memorize.
- Example & Output — runnable queries against a consistent sample schema, with the exact result shown.
- Tips — practical advice that signals strong SQL fluency to interviewers.
- Common Mistakes — the errors that most often cost candidates marks or interview points.
- Interview Angle — how the topic is typically tested and what the panel is really probing for.

Sample Schema Used Throughout This Document

Unless stated otherwise, all examples use the following three tables. Keep this schema in mind — it will make every query in this document easy to follow.

Table	Columns
Employees	emp_id, emp_name, dept_id, manager_id, salary, hire_date
Departments	dept_id, dept_name, location
Projects	project_id, project_name, emp_id, budget, start_date

Sample Data — Employees

emp_id	emp_name	dept_id	manager_id	salary	hire_date
1	Aarav	10	NULL	95000	2019-03-11
2	Diya	10	1	72000	2020-07-01
3	Kabir	20	1	88000	2018-01-15
4	Meera	20	3	61000	2021-05-20
5	Rohan	30	3	61000	2021-11-02
6	Ishaan	30	3	54000	2022-02-17

Sample Data — Departments

dept_id	dept_name	location
10	Engineering	Bengaluru
20	Data & Analytics	Hyderabad
30	Sales	Delhi

SQL Learning Roadmap — One-Page View

If you're starting from scratch, don't jump straight to window functions on day one — most people burn out that way. Follow this rough order instead; it mirrors how the topics actually build on each other, and it's roughly how this handbook is organized too.

Stage	Focus Area	What 'Done' Looks Like
1	SELECT, WHERE, ORDER BY, filtering	You can pull and filter data from a single table without hesitating.
2	Joins (all types) + GROUP BY/HAVING	You can combine 2-3 tables and summarize the result correctly.
3	Subqueries + CTEs	You can break a complex question into smaller named steps.
4	Window functions	You can rank, compare row-to-row, and build running totals confidently.
5	Indexes + transactions	You understand why a query is slow and how concurrent writes are kept safe.
6	Optimization + mock interviews	You can explain your query choices out loud, not just write them.



Placement Insight: Don't try to memorize every function on day one. Get comfortable with Stages 1-3 first — that alone clears the SQL round at most service-based companies. Stages 4-6 are what separate you in product-based company interviews.

Chapter 1 — SQL Fundamentals

1.1 What Is SQL?

SQL (Structured Query Language) is the standard language used to create, query, update, and manage data held in a relational database. It is declarative — you describe what data you want, not how to fetch it; the database engine's query optimizer decides the retrieval strategy.

Database vs DBMS vs RDBMS

Term	Meaning
Database	An organized collection of structured data.
DBMS	Software that manages databases (e.g., file-based, hierarchical, relational).
RDBMS	A DBMS based on the relational model — data stored in tables with rows/columns and relationships enforced via keys (MySQL, PostgreSQL, SQL Server, Oracle).

Categories of SQL Commands

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Defines/modifies schema structure
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Modifies data inside tables
DQL	Data Query Language	SELECT	Retrieves data
DCL	Data Control Language	GRANT, REVOKE	Controls access/permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manages transactions

 **Interview Tip:** Interviewers frequently ask: 'Is TRUNCATE a DDL or DML command, and why does it not fire triggers like DELETE does?' Answer: TRUNCATE is DDL — it deallocates data pages directly and cannot be rolled back in most engines (MySQL), unlike DELETE which is DML and logged row-by-row.

1.2 Basic SELECT Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition
GROUP BY column
HAVING group_condition
ORDER BY column [ASC|DESC]
LIMIT n;                -- MySQL/Postgres
-- SQL Server equivalent: SELECT TOP n ...
```

Logical order of execution (this is the #1 conceptual question asked at every placement interview):

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

💡 Pro Tip: Memorize the execution order above. It instantly explains WHY you can't use a column alias (defined in SELECT) inside a WHERE clause, but CAN use it in ORDER BY — SELECT runs after WHERE but before ORDER BY.

Example

```
SELECT emp_name, salary
FROM Employees
WHERE dept_id = 20
ORDER BY salary DESC;
```

Output:

emp_name	salary
Kabir	88000
Meera	61000

⚠ Common Mistake: Writing `WHERE` after `GROUP BY`, or trying to filter an aggregate with WHERE instead of HAVING. WHERE filters rows before grouping; HAVING filters groups after aggregation.

1.3 Filtering Data

Comparison & Logical Operators

- Comparison: =, != or <>, >, <, >=, <=
- Logical: AND, OR, NOT
- Range: BETWEEN a AND b
- Set membership: IN (v1, v2, ...)
- Pattern matching: LIKE '%pattern%' (_ = single char, % = any sequence)
- Null check: IS NULL / IS NOT NULL (never use = NULL)

```
SELECT emp_name FROM Employees
WHERE salary BETWEEN 60000 AND 90000
AND dept_id IN (10, 20)
AND emp_name LIKE 'A%';
```

⚠ Common Mistake: Using `salary = NULL` to check for nulls. NULL is 'unknown', not a value, so `= NULL` always evaluates to unknown/false. Always use `IS NULL`.

DISTINCT, ORDER BY, LIMIT/OFFSET

```
SELECT DISTINCT dept_id FROM Employees;

SELECT emp_name, salary FROM Employees
ORDER BY salary DESC
LIMIT 3 OFFSET 1;  -- skips the top earner, returns next 3
```

 Key Takeaways

- SQL is declarative — you state what you want, the engine figures out how.
- Logical execution order is FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- WHERE filters rows; it cannot see aggregates or SELECT-clause aliases.
- NULL is 'unknown' — always test it with IS NULL / IS NOT NULL, never with =.

Chapter 2 — Joins

Joins combine rows from two or more tables based on a related column. This is the single most-tested SQL topic in placement interviews.

Types of Joins — Quick Reference

Join Type	Returns
INNER JOIN	Only matching rows in both tables
LEFT JOIN	All rows from the left table + matches from right (NULL if none)
RIGHT JOIN	All rows from the right table + matches from left (NULL if none)
FULL OUTER JOIN	All rows from both tables; NULL where no match exists
CROSS JOIN	Cartesian product — every row of A paired with every row of B
SELF JOIN	A table joined with itself, typically for hierarchical data

Syntax & Example

```
SELECT e.emp_name, d.dept_name
FROM Employees e
INNER JOIN Departments d
    ON e.dept_id = d.dept_id;
```

Output (all 6 employees match a department, so INNER and LEFT give the same result here):

emp_name	dept_name
Aarav	Engineering
Diya	Engineering
Kabir	Data & Analytics
Meera	Data & Analytics
Rohan	Sales
Ishaan	Sales

Self Join — Employee & Manager

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM Employees e
LEFT JOIN Employees m
    ON e.manager_id = m.emp_id;
```

Output:

employee	manager
Aarav	NULL
Diya	Aarav

employee	manager
Kabir	Aarav
Meera	Kabir
Rohan	Kabir
Ishaan	Kabir

 **Interview Tip:** Classic trap question: 'Find employees who do NOT belong to any department' or 'departments with zero employees.' The answer always uses a LEFT JOIN + `WHERE right\_table.key IS NULL` — this pattern is called an anti-join.

```
-- Departments with no employees
SELECT d.dept_name
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

 **Common Mistake:** Confusing WHERE-clause filtering with ON-clause filtering in an OUTER JOIN. Putting a right-table condition in WHERE silently converts your LEFT JOIN into an INNER JOIN, because WHERE runs after the join and drops the NULL-padded rows.

Key Takeaways

- INNER JOIN keeps only matches; LEFT/RIGHT JOIN keep all rows from one side and pad the rest with NULL.
- A self join is just a normal join where both sides happen to be the same table, aliased differently.
- The 'LEFT JOIN + WHERE right.key IS NULL' anti-join pattern is the standard way to find unmatched rows.
- Filter conditions on the right table belong in ON for outer joins, not in WHERE.

Chapter 3 — Aggregation: GROUP BY & HAVING

Aggregate Functions

- COUNT(*) / COUNT(column) — counts rows (COUNT(column) ignores NULLs)
- SUM(), AVG(), MIN(), MAX()

```
SELECT dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM Employees
GROUP BY dept_id
HAVING COUNT(*) > 1
ORDER BY avg_salary DESC;
```

Output (sorted by avg_salary, highest first — Engineering pays the most on average, even though all three departments have exactly two people here):

dept_id	headcount	avg_salary
10	2	83500
20	2	74500
30	2	57500

 **Pro Tip:** Every non-aggregated column in SELECT must appear in GROUP BY (strict SQL mode). MySQL is lenient by default, but relying on that is fragile and considered bad practice.

 **Common Mistake:** Using WHERE to filter on an aggregate, e.g. `WHERE COUNT(\*) > 1` — this throws an error because WHERE runs before aggregation exists. Use HAVING instead.

Key Takeaways

- GROUP BY collapses rows into groups; aggregate functions then summarize each group.
- HAVING filters groups after aggregation — WHERE can't, because it runs first.
- COUNT(column) skips NULLs; COUNT(*) counts every row regardless.
- Every plain column in SELECT must also appear in GROUP BY.

Chapter 4 — Subqueries

Types of Subqueries

- Scalar subquery — returns a single value
- Row/Column subquery — returns one row or one column of multiple values (used with IN)
- Correlated subquery — references the outer query, re-evaluated per outer row
- EXISTS / NOT EXISTS — checks for existence of any matching row (often faster than IN with large sets)

Example — Employees earning above their department's average

```
SELECT e.emp_name, e.salary, e.dept_id
FROM Employees e
WHERE e.salary > (
    SELECT AVG(e2.salary) FROM Employees e2
    WHERE e2.dept_id = e.dept_id      -- correlated: depends on outer row
);
```

Output (Rohan makes the cut too — his department's average is only 57,500, and his salary is 61,000):

emp_name	salary	dept_id
Aarav	95000	10
Kabir	88000	20
Rohan	61000	30

IN vs EXISTS

```
-- IN: materializes the subquery's result set first
SELECT dept_name FROM Departments
WHERE dept_id IN (SELECT dept_id FROM Employees WHERE salary > 80000);

-- EXISTS: stops at the first match per outer row (often better on large/correlated
data)
SELECT dept_name FROM Departments d
WHERE EXISTS (
    SELECT 1 FROM Employees e WHERE e.dept_id = d.dept_id AND e.salary > 80000
);
```

 **Interview Tip:** Be ready to explain: 'IN' can behave unexpectedly with NULLs (NOT IN with a NULL in the list returns zero rows), whereas 'NOT EXISTS' handles NULLs correctly. This NULL trap is a favorite gotcha question.

Key Takeaways

- A correlated subquery re-runs once per outer row; a plain subquery runs once, total.
- EXISTS/NOT EXISTS are usually safer and faster than IN/NOT IN for large or correlated checks.
- NOT IN silently breaks if the subquery's result contains a NULL — NOT EXISTS doesn't have this problem.

Chapter 5 — Common Table Expressions (CTE)

A CTE is a named, temporary result set defined with WITH, scoped to a single statement. CTEs improve readability over nested subqueries and are required for recursion.

Syntax

```
WITH cte_name (col1, col2, ...) AS (  
    SELECT ...  
)  
SELECT * FROM cte_name;
```

Example — Simplifying a Nested Subquery

```
WITH dept_avg AS (  
    SELECT dept_id, AVG(salary) AS avg_sal  
    FROM Employees  
    GROUP BY dept_id  
)  
SELECT e.emp_name, e.salary, d.avg_sal  
FROM Employees e  
JOIN dept_avg d ON e.dept_id = d.dept_id  
WHERE e.salary > d.avg_sal;
```

Recursive CTE — Employee Hierarchy Under a Manager

```
WITH RECURSIVE org_chart AS (  
    -- Anchor member: the top manager  
    SELECT emp_id, emp_name, manager_id, 1 AS level  
    FROM Employees WHERE emp_id = 1  
  
    UNION ALL  
  
    -- Recursive member: find direct reports of the previous level  
    SELECT e.emp_id, e.emp_name, e.manager_id, oc.level + 1  
    FROM Employees e  
    JOIN org_chart oc ON e.manager_id = oc.emp_id  
)  
SELECT * FROM org_chart ORDER BY level;
```

Output:

emp_id	emp_name	manager_id	level
1	Aarav	NULL	1
2	Diya	1	2
3	Kabir	1	2
4	Meera	3	3
5	Rohan	3	3

emp_id	emp_name	manager_id	level
6	Ishaan	3	3

💡 **Pro Tip:** SQL Server uses `WITH RECURSIVE_NAME AS (...)` without the `RECURSIVE` keyword; MySQL 8+/Postgres require `WITH RECURSIVE`. Always mention this dialect difference in interviews — it signals real hands-on experience.

⚠️ **Common Mistake:** Forgetting a terminating condition in the recursive member, causing infinite recursion. Always ensure the recursive term converges (e.g., the join naturally runs out of matching child rows), and set `MAXRECURSION` limits in production.

Key Takeaways

- A CTE is just a named subquery, scoped to one statement — think of it as a readability upgrade.
- Recursive CTEs need an anchor member and a recursive member joined with `UNION ALL`.
- Always make sure the recursive member eventually runs out of matches, or you'll recurse forever.
- Most optimizers inline a non-recursive CTE, so it rarely costs you performance versus a subquery.

Chapter 6 — Window Functions

Window functions perform calculations across a set of rows related to the current row — WITHOUT collapsing rows like GROUP BY does. This is the topic that most separates strong candidates from average ones.

Syntax

```
function_name(...) OVER (
  [PARTITION BY column]
  [ORDER BY column]
  [ROWS/RANGE BETWEEN ... AND ...]
)
```

Ranking Functions

Function	Behaviour on ties
ROW_NUMBER()	Always unique, sequential (1,2,3,4...) even for ties
RANK()	Ties get the same rank; next rank skips (1,1,3,4)
DENSE_RANK()	Ties get same rank; no gap (1,1,2,3)
NTILE(n)	Splits rows into n roughly equal buckets

```
SELECT emp_name, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
FROM Employees;
```

Output:

emp_name	dept_id	salary	rnk	rn
Aarav	10	95000	1	1
Diya	10	72000	2	2
Kabir	20	88000	1	1
Meera	20	61000	2	2
Rohan	30	61000	1	1
Ishaan	30	54000	2	2

Nth Highest Salary (Classic Interview Problem) — solved cleanly with a window function

```
SELECT emp_name, salary FROM (
  SELECT emp_name, salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
  FROM Employees
) ranked
WHERE drnk = 2;    -- change to N for the Nth highest
```

 **Interview Tip:** This exact query — 'find the Nth highest salary' — is asked in nearly every SQL interview, including Microsoft, Amazon, and most service-based companies. Use `DENSE_RANK`, not `RANK`, so duplicate top salaries don't skip a rank.

Offset / Value Functions

- `LAG(col, n)` — value from `n` rows BEFORE the current row
- `LEAD(col, n)` — value from `n` rows AFTER the current row
- `FIRST_VALUE(col)`, `LAST_VALUE(col)` — first/last value in the window frame

```
SELECT emp_name, hire_date,  
       LAG(hire_date) OVER (ORDER BY hire_date) AS prev_hire,  
       LEAD(hire_date) OVER (ORDER BY hire_date) AS next_hire  
FROM Employees;
```

Running Totals & Moving Averages (Aggregate Window Functions)

```
SELECT emp_name, hire_date, salary,  
       SUM(salary) OVER (ORDER BY hire_date  
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,  
       AVG(salary) OVER (ORDER BY hire_date  
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg_3  
FROM Employees;
```

 **Pro Tip:** The default frame when `ORDER BY` is present but no `ROWS/RANGE` is specified is `'RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW'` — this is what silently gives you a running total instead of a full-partition sum. Always specify the frame explicitly for aggregate window functions.

 **Common Mistake:** Trying to filter directly on a window function's result in `WHERE`: `'WHERE RANK() OVER (...) = 1'` is invalid SQL — window functions run after `WHERE`. You must wrap it in a subquery/CTE and filter in the outer query, as shown in the Nth-salary example above.

Key Takeaways

- Window functions calculate across related rows without collapsing them, unlike `GROUP BY`.
- `ROW_NUMBER` = always unique. `RANK` = skips after ties. `DENSE_RANK` = no gaps after ties.
- The Nth-highest-salary pattern (`DENSE_RANK` + filter in an outer query) is the single most-asked SQL interview question — know it cold.
- You cannot filter a window function's result in `WHERE`; wrap it in a subquery or CTE first.

Chapter 7 — Indexes

7.1 What Is an Index?

An index is a separate on-disk data structure (most commonly a B-Tree) that stores a sorted copy of one or more columns plus a pointer back to the full row. It lets the engine locate rows without scanning the entire table — trading extra storage and slower writes for much faster reads.

Clustered vs Non-Clustered

Aspect	Clustered Index	Non-Clustered Index
Data storage	Table data physically sorted in index order	Separate structure; stores a pointer/row-id back to the data
Count per table	Only one (data can only be sorted one way)	Many allowed
Typical use	Primary key by default	Foreign keys, frequent WHERE/JOIN/ORDER BY columns
Lookup cost	Direct — the leaf node IS the data	Extra 'bookmark lookup' hop to fetch full row

Syntax

```
CREATE INDEX idx_emp_dept ON Employees(dept_id);
CREATE UNIQUE INDEX idx_emp_email ON Employees(email);
CREATE INDEX idx_emp_dept_sal ON Employees(dept_id, salary); -- composite index
DROP INDEX idx_emp_dept ON Employees; -- syntax varies by RDBMS
```

How a B-Tree Index Speeds Up Lookups

A B-Tree keeps keys sorted and balanced so any lookup, insert, or delete takes $O(\log n)$ comparisons instead of an $O(n)$ full table scan. Range queries (BETWEEN, <, >, ORDER BY) benefit hugely because the leaf nodes are linked in sorted order.

Composite Index & the Leftmost Prefix Rule

An index on (dept_id, salary) can be used to filter on dept_id alone, or on dept_id AND salary together — but NOT on salary alone. The engine can only use a composite index starting from its leftmost column.

 **Interview Tip:** A guaranteed interview question: 'You have an index on (A, B). Will a query filtering only on B use it?' Answer: No (with rare exceptions like index-only/skip-scans in some engines) — because of the leftmost prefix rule.

 **Pro Tip:** Index columns used in WHERE, JOIN ON, and ORDER BY. Do NOT over-index — every additional index slows down INSERT/UPDATE/DELETE because the index must be maintained on every write.

 **Common Mistake:** Wrapping an indexed column in a function in WHERE, e.g. `WHERE YEAR(hire\_date) = 2021`. This makes the column non-sargable — the optimizer can't use the index and falls back to a full scan. Rewrite as a range: `WHERE hire\_date >= '2021-01-01' AND hire\_date < '2022-01-01'`.

 Key Takeaways

- An index trades write speed and storage for much faster reads — it's not free.
- Only one clustered index per table; as many non-clustered indexes as make sense for your query patterns.
- A composite index only helps if your query uses its leftmost column(s) first.
- Keep predicates sargable — don't wrap an indexed column in a function or implicit cast.

Chapter 8 — Transactions & ACID

8.1 ACID Properties

Property	Meaning
Atomicity	A transaction is all-or-nothing — either every statement commits, or none do.
Consistency	A transaction moves the database from one valid state to another, respecting all constraints.
Isolation	Concurrent transactions don't interfere with each other's intermediate state.
Durability	Once committed, changes survive crashes/power loss (written to durable storage/WAL).

Syntax

```
BEGIN TRANSACTION;

UPDATE Accounts SET balance = balance - 5000 WHERE acc_id = 1;
UPDATE Accounts SET balance = balance + 5000 WHERE acc_id = 2;

-- if anything looks wrong:
-- ROLLBACK;

COMMIT;
```

SAVEPOINT — Partial Rollback

```
BEGIN TRANSACTION;
UPDATE Employees SET salary = salary * 1.1 WHERE dept_id = 10;
SAVEPOINT before_bonus;
UPDATE Employees SET salary = salary + 1000 WHERE dept_id = 20;
ROLLBACK TO before_bonus; -- undoes only the dept_id = 20 update
COMMIT;
```

8.2 Isolation Levels & Concurrency Problems

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

- Dirty Read — reading data another transaction has written but not yet committed.
- Non-Repeatable Read — re-reading the same row within a transaction gives a different value because another committed transaction changed it.
- Phantom Read — re-running the same range query returns a different SET of rows because another transaction inserted/deleted matching rows.

 **Interview Tip:** Expect: 'What isolation level does your database default to, and what trade-off does it make?' MySQL InnoDB defaults to REPEATABLE READ; PostgreSQL and Oracle default to READ COMMITTED. Higher isolation = more consistency but more locking/lower concurrency.

8.3 Locking Deadlock Example

A deadlock occurs when Transaction A holds a lock Transaction B needs, and B holds a lock A needs — neither can proceed. The database detects this cycle and kills one transaction (the 'victim') to break it.

 **Common Mistake:** Always acquiring locks/updating tables in an inconsistent order across different parts of the application. Standardize the order in which tables/rows are touched to avoid deadlocks.

 **Remember:** Higher isolation levels aren't automatically 'better' — they trade concurrency for consistency. Pick the lowest level that still gives your application the guarantees it actually needs.

Key Takeaways

- ACID = Atomicity, Consistency, Isolation, Durability — the four guarantees a transaction gives you.
- COMMIT makes changes permanent; ROLLBACK undoes them; SAVEPOINT allows a partial rollback.
- Isolation level controls the trade-off between consistency and concurrency — know the default for your RDBMS.
- A deadlock is a circular lock wait; consistent lock ordering and short transactions are the main defenses.

Chapter 9 — Query Optimization

9.1 Reading an Execution Plan

```
EXPLAIN SELECT * FROM Employees WHERE dept_id = 20;      -- MySQL/Postgres
EXPLAIN ANALYZE SELECT ...;                               -- Postgres: adds real timing
```

Look for: Seq Scan / Full Table Scan (bad on large tables) vs Index Scan / Index Seek (good); estimated vs actual row counts; and the most expensive node in the plan tree.

9.2 High-Impact Optimization Techniques

1. Select only needed columns — avoid `SELECT *`, which prevents covering-index optimizations and wastes I/O and network bandwidth.
2. Filter early — push `WHERE` conditions as tight as possible; avoid filtering after a large `JOIN` when you could filter before it.
3. Make predicates sargable — avoid functions/type-casts on indexed columns in `WHERE` (see Chapter 7).
4. Use `EXISTS` instead of `IN` for large/correlated subqueries.
5. Add appropriate indexes on `JOIN` keys and frequently filtered/sorted columns — but avoid redundant/unused indexes.
6. Batch large writes — break a huge `UPDATE/DELETE` into smaller chunks to reduce lock contention and transaction log growth.
7. Avoid `N+1` query patterns — fetch related data with a single `JOIN` instead of one query per row in application code.
8. Use appropriate data types — e.g., `INT` over `VARCHAR` for IDs; smaller types mean smaller indexes and more rows per I/O page.
9. Denormalize selectively for read-heavy workloads — trade some redundancy for fewer joins, only where profiling justifies it.
10. Analyze/update statistics regularly — the optimizer's row-count estimates depend on fresh statistics; stale stats produce bad plans.

Example — Rewriting a Non-Sargable Query

```
-- Before (index on hire_date cannot be used)
SELECT * FROM Employees WHERE YEAR(hire_date) = 2021;

-- After (sargable — uses the index as a range scan)
SELECT * FROM Employees
WHERE hire_date >= '2021-01-01' AND hire_date < '2022-01-01';
```

 **Pro Tip:** In interviews, always tie an optimization back to **WHY** it helps at the storage-engine level (fewer pages read, index used, fewer rows scanned) — that depth is what distinguishes senior-level answers.

 **Common Mistake:** Adding an index and assuming it will always be used. The optimizer may still choose a full scan if the table is small, or if the filtered column has low selectivity (e.g., a boolean flag) where scanning is actually cheaper than random I/O through the index.

 Key Takeaways

- EXPLAIN (or EXPLAIN ANALYZE) is your first move when a query feels slow — read the plan before guessing.
- Sargable predicates, selective indexes, and avoiding SELECT * are the highest-leverage, lowest-effort wins.
- Optimization is a trade-off exercise — more indexes help reads but always cost writes.
- An index existing doesn't guarantee it's used; the optimizer picks whatever plan it estimates is cheapest.

Chapter 10 — Normalization (Quick Reference)

Normal Form	Rule
1NF	Atomic column values; no repeating groups/arrays in a single column.
2NF	1NF + every non-key column depends on the WHOLE primary key (no partial dependency, relevant for composite keys).
3NF	2NF + no transitive dependency — non-key columns depend only on the key, not on other non-key columns.
BCNF	Stronger 3NF — every determinant is a candidate key.

 **Interview Tip:** Be ready to normalize a sample messy table on a whiteboard, AND to explain when you'd deliberately denormalize (reporting/analytics tables, read-heavy dashboards) for performance.

Key Takeaways

- Each higher normal form removes one more kind of redundancy/dependency issue.
- 3NF is the practical target for most transactional schemas.
- Normalize for data integrity in OLTP systems; denormalize deliberately for read-heavy reporting.

Chapter 11 — Common Mistakes Checklist

- Using WHERE instead of HAVING to filter aggregated results.
- Comparing to NULL with `=` instead of IS NULL / IS NOT NULL.
- Letting a LEFT JOIN silently become an INNER JOIN by filtering the right table in WHERE instead of ON.
- Wrapping indexed columns in functions, breaking sargability.
- Using SELECT * in production queries and ORM-generated code.
- Forgetting GROUP BY must include every non-aggregated selected column.
- Assuming NOT IN behaves like NOT EXISTS when the subquery list can contain NULLs.
- Not specifying the window frame (ROWS/RANGE) and getting a running total instead of a full-partition aggregate, or vice-versa.
- Over-indexing a write-heavy table, hurting INSERT/UPDATE/DELETE throughput.
- Running large UPDATE/DELETE statements in one transaction, causing long lock waits and log bloat.
- Ignoring transaction isolation level implications and assuming reads are always consistent.
- Using implicit joins (comma syntax in FROM) instead of explicit JOIN ... ON — harder to read and easy to turn into an accidental cross join.

Key Takeaways

- Most of these mistakes come from forgetting the logical execution order (Chapter 1) — revisit it if any of these felt surprising.
- Read this list once the night before every interview; it takes under two minutes and catches easy, avoidable losses.

Chapter 12 — Interview Questions & Answers

Curated, high-frequency SQL interview questions, ordered roughly from conceptual to scenario-based.

Q1. What is the difference between DELETE, TRUNCATE, and DROP?

DELETE is DML, removes rows (optionally filtered by WHERE), is logged row-by-row, can be rolled back, and fires triggers. TRUNCATE is DDL, removes all rows by deallocating pages, is minimally logged, faster, and generally cannot be rolled back (varies by engine). DROP removes the entire table structure, including its schema, indexes, and constraints.

Q2. What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping/aggregation happens. HAVING filters groups after GROUP BY has produced aggregated results. You cannot reference an aggregate function in WHERE.

Q3. Explain the difference between UNION and UNION ALL.

UNION combines result sets from two queries and removes duplicate rows (implies a sort/dedup step, so it's slower). UNION ALL keeps all rows including duplicates and is faster since no dedup pass is needed. Use UNION ALL whenever you know duplicates aren't a concern.

Q4. What is a primary key vs a unique key vs a foreign key?

A primary key uniquely identifies each row, disallows NULLs, and a table can have only one. A unique key also enforces uniqueness but allows one NULL (in most engines) and a table can have several. A foreign key enforces referential integrity by referencing a primary/unique key in another (or the same) table.

Q5. What is a correlated subquery, and how is it different from a regular subquery?

A correlated subquery references a column from the outer query, so it must logically be re-evaluated once per outer row, whereas a plain (non-correlated) subquery executes once and its result is reused for every outer row.

Q6. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

ROW_NUMBER() assigns unique sequential numbers even to tied rows. RANK() assigns the same rank to ties but skips subsequent rank values by the count of ties. DENSE_RANK() assigns the same rank to ties without leaving any gap in the ranking sequence.

Q7. How would you find duplicate rows in a table?

Group by the columns that define a duplicate and filter groups with COUNT(*) > 1 using GROUP BY plus HAVING, or use ROW_NUMBER() OVER (PARTITION BY duplicate_columns ORDER BY id) in a CTE and select/delete rows where the row number is greater than 1.

Q8. Write a query to find the second highest salary without using LIMIT/TOP.

SELECT MAX(salary) FROM Employees WHERE salary < (SELECT MAX(salary) FROM Employees); This generalizes poorly to the Nth case, where DENSE_RANK() (shown in Chapter 6) is the cleaner, more maintainable solution.

Q9. What is a clustered index, and how many can a table have?

A clustered index determines the physical storage order of table rows, so only one is allowed per table (data can only be sorted one way at a time). It is created automatically on the primary key in most RDBMS unless specified otherwise.

Q10. What is index sargability, and why does it matter?

Sargable (Search ARGument-ABLE) predicates let the optimizer use an index directly, typically simple comparisons on a raw column. Wrapping a column in a function or performing an implicit type conversion makes the predicate non-sargable, forcing a full scan even when a relevant index exists.

Q11. Explain the ACID properties with a real example.

Using a bank transfer: Atomicity ensures the debit and credit either both happen or neither does; Consistency ensures total balance across accounts stays correct; Isolation ensures a concurrent balance check doesn't see a half-finished transfer; Durability ensures the transfer survives a crash immediately after commit.

Q12. What is a deadlock, and how can it be avoided?

A deadlock happens when two transactions each hold a lock the other needs, creating a cycle with no progress possible; the database detects it and rolls back one transaction. It can be reduced by accessing tables/rows in a consistent order, keeping transactions short, and using appropriate isolation levels.

Q13. What is the difference between a CTE and a subquery?

A CTE (WITH clause) is named, defined once, and can be referenced multiple times in the same statement, and supports recursion; a regular subquery is typically inline, can become deeply nested and harder to read, and cannot recurse. Performance is usually similar since most optimizers inline non-recursive CTEs.

Q14. What is the difference between a CTE and a temporary table?

A CTE exists only for the duration of a single statement and is not materialized as physical storage by default; a temporary table is physically created, persists for the session/transaction, can be indexed, and can be reused across multiple statements.

Q15. How does a LEFT JOIN combined with IS NULL find unmatched rows?

A LEFT JOIN keeps every row from the left table and pads unmatched right-table columns with NULL; filtering WHERE right_table.key IS NULL isolates exactly the left-table rows that had no match, which is the standard anti-join pattern.

Q16. What happens if you GROUP BY a column but SELECT another non-aggregated column not in the GROUP BY list?

Standard SQL raises an error because the value is ambiguous — the engine wouldn't know which row within the group to pick. MySQL's default mode may silently allow it and return an arbitrary row's value, which is why relying on that behavior is discouraged.

Q17. What is denormalization, and when would you use it?

Denormalization intentionally introduces redundancy (e.g., storing a computed total or duplicating a lookup value) to reduce the number of joins needed for frequent reads. It trades some write complexity and storage for read performance, and is common in reporting/analytics/read-heavy systems.

Q18. What is the difference between OLTP and OLAP systems?

OLTP (Online Transaction Processing) systems handle many short read/write transactions with strong normalization, favoring consistency and quick single-row operations (e.g., an order system). OLAP (Online Analytical Processing) systems handle large read-heavy analytical queries over historical data, often denormalized into star/snowflake schemas for fast aggregation.

Q19. How do you find the Nth highest salary per department?

Use `DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)` inside a CTE or subquery, then filter the outer query `WHERE drnk = N`; partitioning resets the ranking separately for each department.

Q20. What is the difference between a covering index and a regular index?

A covering index includes all the columns a query needs (in its key or as included columns), so the engine can satisfy the query entirely from the index without a bookmark lookup back to the base table, which is significantly faster for read-heavy queries.

 Key Takeaways

- Interviewers care as much about how you explain a query as whether it runs — narrate your reasoning.
- A large share of SQL rounds recycle the same 15-20 patterns; Chapter 15 (Most Asked SQL Interview Patterns) collects the highest-frequency ones.
- If you can only revise one thing before an interview, revise this Q&A list.

Chapter 13 — Practice Problems

Attempt each problem using the sample schema from the beginning of this document before checking any reference solution online. Solutions are intentionally left as an exercise — writing the query yourself, under light time pressure, is what builds interview speed.

Foundational

11. List all employees hired after 1st January 2020, ordered by hire date.
12. Find the total salary expense per department.
13. List employees whose name starts with 'A' or ends with 'a'.
14. Find all departments that have at least 2 employees.
15. Retrieve employee names along with their department name and location.

Intermediate

16. Find employees who earn more than their department's average salary.
17. Find the department with the highest average salary.
18. List each manager along with the number of direct reports they have.
19. Find employees who joined in the same year as their manager.
20. Write a query to swap the salary values of the two highest-paid employees (using a single UPDATE).
21. Find all departments that have no employees at all.

Advanced (Window Functions & CTEs)

22. Find the 3rd highest salary in each department using DENSE_RANK().
23. Compute a running total of salaries ordered by hire_date across the whole company.
24. For each employee, show the salary difference from the previous hire (using LAG).
25. Find the top 2 earners per department using RANK() with ties handled correctly.
26. Using a recursive CTE, list the full management chain (all ancestors) for a given employee.
27. Identify duplicate employee records (same name and dept_id) and keep only the earliest hire_date row for each duplicate set.
28. Calculate a 3-month moving average of monthly project budgets (assume a Projects table with start_date and budget).

 **Pro Tip:** When practicing, always say your query out loud as if explaining it to an interviewer — clarity of explanation is scored as heavily as query correctness in most SQL interview rounds.

Key Takeaways

- Solve every problem against the sample schema by hand before looking anything up.

- Time yourself once you're comfortable — most SQL interview rounds expect a working query in 5-10 minutes.
- The Advanced section is where window functions and CTEs from Chapters 5-6 actually get tested — revisit those chapters if you get stuck.

Chapter 14 — Top 25 SQL Functions Cheat Sheet

These are the functions that show up again and again, whether you're writing a query in an interview or cleaning data at work. Skim this the morning of an interview rather than trying to memorize it weeks in advance — it sticks better that way.

#	Function	Category	What It Does
1	COUNT()	Aggregate	Counts rows; COUNT(col) ignores NULLs, COUNT(*) doesn't.
2	SUM()	Aggregate	Adds up numeric values in a column.
3	AVG()	Aggregate	Mean of numeric values, ignoring NULLs.
4	MIN() / MAX()	Aggregate	Smallest / largest value in a column.
5	CONCAT()	String	Joins two or more strings together.
6	SUBSTRING()	String	Extracts part of a string by position and length.
7	LENGTH() / LEN()	String	Number of characters in a string (LEN in SQL Server).
8	UPPER() / LOWER()	String	Converts text to upper/lower case.
9	TRIM()	String	Removes leading/trailing whitespace.
10	REPLACE()	String	Substitutes one substring for another.
11	COALESCE()	Null Handling	Returns the first non-NULL value in a list — works across all major RDBMS.
12	NULLIF(a, b)	Null Handling	Returns NULL if a = b, else returns a — handy to avoid divide-by-zero.
13	IFNULL() / ISNULL()	Null Handling	Two-argument NULL replacement (MySQL / SQL Server flavors of COALESCE).
14	CAST() / CONVERT()	Conversion	Explicitly converts a value from one data type to another.
15	ROUND()	Numeric	Rounds a number to a given number of decimal places.
16	ABS()	Numeric	Absolute (non-negative) value of a number.
17	NOW() / GETDATE()	Date/Time	Current timestamp (NOW in MySQL/Postgres, GETDATE in SQL Server).
18	DATEDIFF()	Date/Time	Difference between two dates (units vary slightly by RDBMS).
19	DATE_ADD() / DATEADD()	Date/Time	Adds an interval (days, months, etc.) to a date.
20	EXTRACT() / DATEPART()	Date/Time	Pulls out a part of a date, like year or month.
21	ROW_NUMBER()	Window	Unique sequential number per row within a partition.
22	RANK() / DENSE_RANK()	Window	Ranking with gaps / without gaps after ties.
23	LAG() / LEAD()	Window	Value from a previous / next row in the ordered set.

#	Function	Category	What It Does
24	CASE WHEN	Conditional	Inline if/else logic inside a SELECT — the backbone of manual pivoting.
25	EXISTS()	Logical	Tests whether a subquery returns any row at all.

 **Pro Tip:** A quick trick: if you can't remember whether a function is 'GETDATE' or 'NOW', just say both out loud and note the dialect — interviewers respect that more than a confident wrong guess.

Key Takeaways

- You don't need all 25 memorized verbatim — recognizing them when you see them is usually enough.
- Null-handling functions (COALESCE, NULLIF, IFNULL/ISNULL) come up constantly in real data cleanup, not just interviews.

Chapter 15 — Most Asked SQL Interview Patterns

Strip away the specific business context, and most SQL interview questions turn out to be one of a small handful of patterns wearing a different costume. Recognize the pattern and the query almost writes itself.

1. Nth Highest Value

Rank the values with `DENSE_RANK()` and filter on the rank in an outer query. Covered in full in Chapter 6.

```
SELECT * FROM (
  SELECT col, DENSE_RANK() OVER (ORDER BY col DESC) AS r
  FROM table_name
) t WHERE r = N;
```

2. Duplicate Detection

Either `GROUP BY` the duplicate-defining columns and filter with `HAVING COUNT(*) > 1`, or tag rows with `ROW_NUMBER()` and keep/remove based on that.

```
SELECT name, dept_id, COUNT(*)
FROM Employees
GROUP BY name, dept_id
HAVING COUNT(*) > 1;
```

3. Top-N Per Group

`PARTITION BY` the grouping column inside a ranking window function, then filter the rank in an outer query — the same skeleton as Nth Highest, just partitioned.

```
SELECT * FROM (
  SELECT *, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
  FROM Employees
) t WHERE r <= 2;
```

4. Anti-Join (Find What's Missing)

`LEFT JOIN` from the table you want everything from, then filter `WHERE` the right side is `NULL`. Covered in Chapter 2.

```
SELECT d.dept_name FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

5. Running Total / Cumulative Sum

An aggregate window function ordered by the relevant column with an explicit frame. Covered in Chapter 6.

```
SELECT emp_name, hire_date,
SUM(salary) OVER (ORDER BY hire_date
```



```
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM Employees;
```

6. Manual Pivot (Rows to Columns)

CASE WHEN inside an aggregate turns row values into columns — no PIVOT keyword needed, and this version works identically across MySQL, Postgres, and SQL Server.

```
SELECT
  SUM(CASE WHEN dept_id = 10 THEN salary ELSE 0 END) AS engineering_total,
  SUM(CASE WHEN dept_id = 20 THEN salary ELSE 0 END) AS data_total,
  SUM(CASE WHEN dept_id = 30 THEN salary ELSE 0 END) AS sales_total
FROM Employees;
```

7. Consecutive Rows / Streaks

Subtract a ROW_NUMBER() from a date or sequence column — rows that belong to the same streak land on the same difference value, which you can then GROUP BY.

8. Median (No Built-in MEDIAN in Most Engines)

Use two window-ranked halves and average the middle value(s), or PERCENTILE_CONT(0.5) where the engine supports it (PostgreSQL and SQL Server do; MySQL does not).

 **Interview Tip:** When you spot one of these eight patterns mid-interview, say so out loud: 'this looks like a Top-N-per-group problem, so I'll partition by X.' It shows pattern recognition, not just query-writing.

Key Takeaways

- Most 'unique' interview questions are one of these eight patterns wearing different table names.
- Naming the pattern out loud before you write the query is a strong signal of experience.

Chapter 16 — 30 One-Line SQL Interview Facts

Rapid-fire facts for last-minute revision — the kind of thing that turns into a confident one-line answer instead of a blank stare.

29. SQL is declarative — you describe the result, the engine decides how to get there.
30. A primary key cannot contain NULL values; a unique key can (in most engines, one NULL).
31. TRUNCATE is DDL and hard to roll back; DELETE is DML, logged, and roll-back-friendly.
32. INNER JOIN returns only rows that match on both sides.
33. HAVING filters after GROUP BY; WHERE filters before it.
34. A clustered index physically orders the table's data; only one is allowed per table.
35. NULL is never equal to NULL — always use IS NULL, not = NULL.
36. UNION removes duplicates and sorts internally; UNION ALL keeps everything and is faster.
37. Views don't store data themselves — they're stored queries (unless materialized).
38. Foreign keys enforce referential integrity between tables.
39. A composite index only helps if the query uses its leftmost column(s) first.
40. ACID stands for Atomicity, Consistency, Isolation, Durability.
41. A deadlock is two transactions waiting on each other's locks in a circle.
42. RANK() skips numbers after a tie; DENSE_RANK() doesn't.
43. ROW_NUMBER() always produces unique values, even across ties.
44. A CTE is scoped to the single statement it's defined in.
45. Recursive CTEs need an anchor member and a recursive member joined with UNION ALL.
46. Window functions calculate per row without collapsing the result set, unlike GROUP BY.
47. A correlated subquery runs once per outer row; a plain subquery runs once, total.
48. EXISTS often beats IN for large or correlated subqueries.
49. Normalization reduces redundancy; denormalization trades redundancy for read speed.
50. OLTP systems handle many small transactions; OLAP systems handle large analytical queries.
51. Indexes speed up reads but slow down every INSERT, UPDATE, and DELETE.
52. A non-sargable predicate (a function wrapped around an indexed column) can't use that index.
53. SELECT * in production code is a habit worth breaking early.
54. MySQL's InnoDB defaults to REPEATABLE READ; PostgreSQL defaults to READ COMMITTED.
55. A view simplifies a complex query but doesn't automatically make it faster.
56. Shorter transactions mean less lock contention and fewer deadlocks.
57. GROUP BY runs before SELECT in the logical order of execution.
58. The best way to revise SQL is to write queries by hand, not just read about them.

Key Takeaways

- These 30 facts compress almost every earlier chapter into one-line answers — read them the morning of your interview.
- If any single line feels unfamiliar, that's your signal to jump back to the matching chapter.

Chapter 17 — MySQL vs PostgreSQL vs SQL Server

Interviewers sometimes ask which database you've used and what's different about it. You don't need deep expertise in all three, but knowing these differences off the top of your head shows real hands-on exposure.

Aspect	MySQL	PostgreSQL	SQL Server
License	Open source (Oracle-owned)	Open source	Commercial (Microsoft)
Default isolation level	REPEATABLE READ	READ COMMITTED	READ COMMITTED
Row limiting	LIMIT n	LIMIT n	TOP n / OFFSET-FETCH
Auto-increment	AUTO_INCREMENT	SERIAL / IDENTITY	IDENTITY(1,1)
String concatenation	CONCAT()	or CONCAT()	+ or CONCAT()
Recursive CTE	WITH RECURSIVE	WITH RECURSIVE	WITH (no RECURSIVE keyword)
JSON support	JSON type + functions	Native JSONB (strong indexing support)	JSON stored as NVARCHAR + functions
Typically strong at	Fast reads, simple web apps	Complex queries, data integrity, extensibility	Enterprise/Windows & .NET ecosystems

 **Placement Insight:** You won't be expected to be an expert in all three. What matters is being able to say 'I've mainly worked with X, but I know Y differs in Z' — that one sentence usually satisfies the question.

Key Takeaways

- Core SQL concepts transfer across all three engines — only syntax and a few defaults change.
- Know your primary database's isolation-level default and row-limiting syntax cold.

Chapter 18 — Common SQL Interview Mistakes (Beyond the Query Itself)

Chapter 11 covered technical mistakes inside the SQL itself. This chapter is about the mistakes candidates make in how they approach the interview — just as costly, and much easier to fix once you notice them.

- Writing code immediately without clarifying the question — 'find duplicates' means different things depending on which columns define a duplicate.
- Not asking whether NULLs, ties, or empty tables need special handling before writing the query.
- Reaching for three nested subqueries when a single JOIN would do the job more simply.
- Going silent while thinking — interviewers can't score reasoning they can't hear.
- Not mentioning performance at all until directly asked 'how would you optimize this?'
- Mixing up WHERE and HAVING under pressure, even after knowing the difference cold in practice.
- Declaring the query 'done' without mentally tracing it against a small example first.
- Defaulting to SELECT * out of habit, even in an interview setting where every column should be intentional.
- Not stating assumptions out loud when a question is ambiguous (e.g., 'I'll assume duplicate means same email').
- Panicking on syntax instead of explaining the logic — most interviewers care more about approach than perfect semicolons.



Remember: A wrong-but-well-reasoned query, explained clearly, usually scores better than a correct query delivered with silence and hesitation.



Key Takeaways

- Most of these mistakes are about communication, not SQL knowledge — they're fixable in a single mock interview.
- Clarify, narrate, and check your own query before calling it final.

Chapter 19 — Placement Revision Checklist

A tick-box list for the last 48 hours before an interview. Go top to bottom — if something doesn't feel automatic, jump back to that chapter.

- ☐ Logical execution order (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT)
- ☐ All join types, plus the LEFT JOIN + IS NULL anti-join pattern
- ☐ GROUP BY / HAVING and the difference from WHERE
- ☐ Correlated subqueries vs plain subqueries, and IN vs EXISTS
- ☐ CTEs, including writing one recursive CTE from scratch
- ☐ Window functions — ROW_NUMBER, RANK, DENSE_RANK, LAG/LEAD, running totals
- ☐ The Nth-highest-salary pattern, both overall and per group
- ☐ Clustered vs non-clustered indexes and the leftmost prefix rule
- ☐ ACID properties and at least one isolation-level trade-off
- ☐ Three optimization techniques you can explain with a 'why', not just a 'what'
- ☐ Normalization up to 3NF, and one real reason to denormalize
- ☐ Practiced explaining at least five queries out loud, not just writing them silently

 **Placement Insight:** Print or screenshot just this checklist page — it's the fastest possible revision pass and covers roughly 80% of what actually gets asked.

Appendix — One-Page Cheat Sheet

Execution Order

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

Joins

```
INNER JOIN    -- matches only
LEFT JOIN     -- all left + matches
RIGHT JOIN    -- all right + matches
FULL OUTER   -- all rows both sides
CROSS JOIN    -- cartesian product
```

Window Function Skeleton

```
func(...) OVER (PARTITION BY col ORDER BY col ROWS BETWEEN ... AND ...)
```

Transaction Skeleton

```
BEGIN TRANSACTION; ... COMMIT; -- or ROLLBACK;
```

Index Rule of Thumb

Index columns in WHERE / JOIN / ORDER BY. Keep predicates sargable. Composite index usable left-to-right only.

End of Notes — Good luck with your placements!

Created by Ansh Banga

LinkedIn: <https://www.linkedin.com/in/ansh-banga-775275330/>